# Extending Multi-Task Learn-to-Learn using a T5 Transformer and Graph Neural Networks

Cole Hoffer, Elizabeth Weeks, Sanja Simonovikj

## 1. Introduction

In this project we extend the work done in Measuring Massive Multitask Language Understanding (Hendrycks et al., 2020), which offers a new benchmark to comprehensively evaluate model's academic and professional understanding on 57 different tasks including elementary mathematics, US history, computer science, law, and more. Additionally, we curate a new dataset and look to solve certain types of machine-learning questions from 6.036, the introductory Machine Learning course at MIT. In particular, we try the following tasks:

1. Use a T5 transformer architecture (Raffel et al., 2019) instead of GPT-3 (Brown et al., 2020) on existing dataset benchmarks.

2. Solve six types of 6.036 (Intro to Machine Learning at MIT) questions using T5 transformers and Graph Neural Networks.

3. Consider several heuristic approaches and task setups and their comparative performances.

Initially, we compare our results of using T5 to the results using the GPT-3 model seen in the recent "Measuring Massive Multitask Language Understanding" paper (Hendrycks et al., 2020) (Brown et al., 2020). We also consider a heuristic approach using DistilBert QA from Huggingface (Wolf et al., 2020a). From there as an extension, we looked at text-based math questions where the answer is a numerical value which can be obtained upon evaluation of an *expression tree*. For this we use a T5 transformer with a GNN (Graph Neural Networks). We tried to solve certain types of math questions from the benchmark using this method and also we analyze the failure of the model to answer many of these questions. Finally, as our main contribution, we curate a set of 1200 6.036 questions from six machine learning based question categories and we use it in our T5 + Graph Neural Network setup, where we observe up to a 89% overall test set accuracy.

## 2. Background and Related Work

In recent years the dominant paradigm in NLP has been to pretrain large models on massive text corpora obtained from various readily available sources, providing the model a broad range of knowledge. Later these models are finetuned on a smaller, specialized dataset, making the pretrained model useful in a multitude of downstream tasks. Recent advances in transformer architectures have allowed a large pre-trained language model to be used for few-shot learning as well. Such is the case with GPT-3 (Brown et al., 2020), an autoregressive language model with 175 billion parameters, 10x more than any previous non-sparse language model.

To measure the success of these massive models in the few-shot learning scenario a very recent benchmark has been proposed by Hendricks et al. (Hendrycks et al., 2020), where unlike the current benchmarks that measure the commonsense or narrow linguistic understanding underlying the language models, the new test seeks to "measure arbitrary real-world text understanding" and "comprehensively evaluate the breadth and depth of a model's academic and professional understanding." The massive multitask test comprises multiple-choice questions at different levels of difficulty from various branches of knowledge, divided into a few-shot development set, a validation set, and a test set. Each of the 57 subjects contains at least 100 test examples to examine the models in zero-shot and few-shot settings. In the experiments in the paper, all models ranked below expert-level performance for all tasks. The largest GPT-3 model had the best performance, scoring an average of 43.9 percent accuracy, improving over random chance by about 20 percentage points. We want to use this benchmark to test alternative available transformer architectures such as T5 (Raffel et al., 2019) to check whether and by how much simpler transformer models perform worse than massive models such as GPT-3.

While large Transformers perform with nearly 70% accuracy on topics such as foreign policy, Psychology, and Marketing, few-shot Transformer performance on STEM related topics such as Chemistry, Physics or Math are currently slightly above random (Hendrycks et al., 2020) . Recent work (Chen et al., 2020), (Koncel-Kedziorski et al., 2016) targeting these tasks enhance the Transformer with a GNN to predict an

expression tree whose evaluation solves these problems. Graph neural networks (GNN's) can be used to predict graph properties such as nodes, edges or full graph encodings. In our case, we use graph convolutional network (GCN) (Kipf & Welling, 2017) for predicting an encoding of a tree which represents a mathematical expression. (Zheng et al., 2020) uses a graph representing the relationships between words and quantities, and a graph representing the relationships between different quantities in the input question. We use an implementation based on (Zheng et al., 2020) which we also got a chance to use in our exercise 2 with the MAWPS dataset (Koncel-Kedziorski et al., 2016) consisting of 2,373 math word problems.

## 3. Methods

### 3.1. Replacement of GPT-3 with T5

We replace GPT-3 with the T5 transformer (Text-To-Text Transfer Transformer) in (Hendrycks et al., 2020). The T5 model consists of an encoder and a decoder. The encoder contains multiple segments of self-attention layers and small feed forward networks. An input sequence that is mapped to an embedding sequence is passed into this encoder. Layer initialization is used and then the sub-components inputs and outputs are summed and dropout is used for each step. The decoder is similar to the encoder but the self-attention layers include autoregression or casual self-attention.

The T5 model was tested on a large variety of downstream language tasks with varying success which is what leads us to use T5 for our downstream task. In order to use the T5 model all tasks must be in a text-to-text format. The questions used for the 57 academic subjects from (Hendrycks et al., 2020) are already in this format since they are language tasks that provide context for the classifier and have a text based answer.

### 3.2. T5 Transformer and GNN's

As in (Zhang et al., 2020), the model consists of an encoder which outputs a representation for the input question, a GNN which encodes the relationship between the entities and quantities, and a tree decoder which generates the desired expression using the enhanced representation. Importantly, we use a pre-trained T5 transformer (small) as the base language model. We note that using "base" did not produce significant changes in performance. As in the original paper we use a graph convolution network (GCN) to encode relationships among entities and quantities. The model architecture and hyper-parameters are given below:

- Encoder: we initialize the encoder to the pre-trained encoder of a T5-small model obtained from the Huggingface Transformers library (Wolf et al., 2020a). This model consists of 6 transformer-based layers with a model width of 512.
- GNN: we use a GCN with 4 parallel heads and dropout of 0.5. We obtain the graph convolution modules from the Deep Graph Library (DGL) (Wang et al., 2020; Zheng et al., 2020).
- Decoder: we use a tree decoder with dropout rate of 0.5. To generate an expression, we use a beam size of 5 for the beam search.

We train the model for 100 epochs using Adam optimizer with a learning rate of 1e-3, weight decay of 1e-5, batch size of 64, and cosine annealing. We note that this is the setup from exercise 2 in our Meta-Learning course and we will refer to it as "exercise 2" approach.

### 3.3. DistilBert QA with Heuristics

One of our attempts tackle the multiple choice question task from (Hendrycks et al., 2020) is a heuristic approach which uses the default Question-answering pipeline from HuggingFace (Wolf et al., 2020b), which uses the DistilBert model (Sanh et al., 2020). The interface function takes in a question and a context and outputs a generated answer in free form. We use the four multiple choice options as the context. Note that this is a zero-shot approach as we do not include other examples in the input and do not finetune the model on our task. To evaluate we find the multiple choice answer that is the closest match with the generated answer. To do this we calculate the LCS (longest common substring) of the prediction and a multiple choice option and divide by the maximum length of the two. We consider our final answer to be the choice with the largest similarity to our prediction. We then compare this answer with the ground truth answer to determine success.

### 3.4. Other Approaches That Did Not Work Well

#### 3.4.1. Combining Learn to Learn with Exercise 2

In this approach we wanted to see if math-based questions from (Hendrycks et al., 2020) could be used with the setup in Section 3.2. We take 5 math subjects from the (Hendrycks et al., 2020) benchmark and we filter the questions to get only those that contain numerical values in the body of the question and the ground truth answer is a numerical value. We form question-answer pairs transforming the multiple-choice question into a free numerical answer question. The 5 subjects are: elementary math, high school math, high school statistics, college math and abstract algebra. These questions often require knowledge about different mathematical concepts ranging from elementary to advanced i.e. they are not purely computational. We used this dataset purely for evaluation purposes, while we retain the train

data from (Koncel-Kedziorski et al., 2016). The final test accuracy ended up being 4%. We attribute this failure to two reasons. First, the model lacks math knowledge as the train data does not include any knowledge about math or statistics, and neither does the T5 language model, so it is not surprising that the model fails when presented with knowledge-based questions. Secondly, the model is best suited for computational math questions where there is an underlying equation to be solved, in similar spirit to the math word problems in the training data. This is not the type of data we tested with in this experiment, so the low accuracy is understandable.

This experiment raises the question of how much can models such as our T5 + GNN generalize to different types of math questions, such as those in (Hendrycks et al., 2020) and even many types of machine learning coursework questions that have varying structures and answer types. Further research can consider if adding math knowledge in the pretraining of the language models would help in tackling these problems.

#### 3.4.2. Increased K-shots

One modification we looked at in the beginning of the project was using the T5 base model and including 0 and/or k-shot learning. We looked at this approach in the context of replacing a GPT-3 model with a T5 transformer as discussed in Section's 3.1 and 5.3. We first started by experimenting with k-shot learning in the T5-base model, and then had planned to apply it to the larger T5 models if the T5-base was successful.

We soon discovered that as the number of training shots increased, the accuracy of our predictions decreased. Our hypothesis for why this occurred is that the model began to over-fit to the few shots of training data. This can be seen in some of the math problems which have true/false answers pairs. The model begins to select the same combination for all of math questions that have that answer setup. We tried implementing a meta-learning training scheme similar from OpenAI called Reptile (what is used in GPT-3 as well) (Nichol et al., 2018). At a high level, it has an outer and inner training loop, where the outer looks to train on general knowledge while the inner can fine tune on a very specific question type, enabling very successful k-shot learning. But as the results in Figure 1 show, it does not to be working as expected. We also tried simply adding k training shots in a few-epoch fine tuning training cycle (no setup like Reptile or MAML), and experienced similar results which further disuaded us from using this technique in our later work in the project.

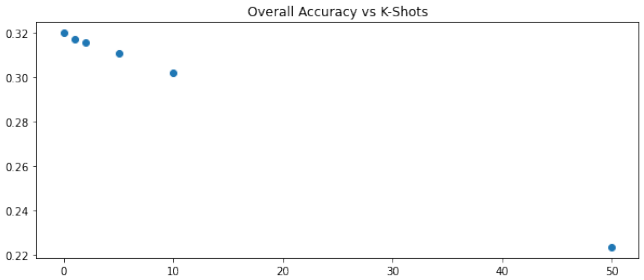


*Figure 1.* Eval accuracy as the number of training shots increased when looking at academic subject data from the Hendrycks et al. paper where we replaced the GPT-3 model with a pretrained T5 model.

## 4. Data

Throughout the project, we used three different datasets/setups to evaluate our models, which are described in detail below. Note that we use overall prediction accuracy as our main evaluation metric (classification accuracy for multiple choice).

### 4.1. Questions from Intro to ML (6.036)

6.036 is the Introduction to Machine Learning course at MIT and is available to students on 6.036 MITx. Using the various weekly exercises, we quickly realized there were not enough unique questions on the web portal to generate a sufficiently large dataset. Thus we decided to pull specific questions from MITx which could be converted into having a single numeric answer, and then created many permutations using these question templates in order to increase the size of the training and testing data. We did not use a transformer to vary the wording of the generated questions but this is a possible extension that would make our dataset more robust.

We created 200 questions per category, resulting in 1200 questions in total. We set the train/test split ratio to be 2 to 1. While we found the dataset size to be sufficient for our purposes, we note that it can be increased by simply adding more values to be inserted in our question-generating template. Below we describe each of the 6 types of 6.036 questions we looked at.

#### 4.1.1. Logistic Regression

The first type of question we took is modeled off of the week 4 exercise 1.1, where students are to find particular logistic regression values based on provided input vectors. The question we wrote is "x is ( x_a , x_b ) , theta is ( theta_a , theta_b ) and theta_0 is theta_0 . What is the value of theta times x plus theta_0 ?" For the question generation the

values of x, theta, and theta_0 varied to create train and test data. The generated expression (used for training data) is "$((x_a * theta_a) + (x_b * theta_b)) + theta_0$".

| | |
|---|---|
| **Question** | x is ( {x_a} , {x_b} ) , theta is ( {theta_a} , {theta_b} ) and theta_0 is {theta_0} . What is the value of theta times x plus theta_0 ? |
| **Expression** | (({x_a}*{theta_a})+({x_b}*{theta_b}))+{theta_0} |
| **Ex Question** | x is ( 1 , 2 ) , theta is ( 1 , 0 ) and theta_0 is 0.5 . What is the value of theta times x plus theta_0 ? |
| **Ex Expression** | ((1*2)+(1*0)+0.5) |
| **Ex Answer** | 2.5 |

*Figure 2.* The 6.036 logistic regression question template with a concrete example provided.

#### 4.1.2. Gradient Descent 1

The first gradient descent question is based off of week 4 exercise 2.1. The question is "if f(theta) is c1 times theta plus c2 squared and theta is theta what is f(theta) ?" where theta, c1, and c2 all vary. The expression comes from "$(c1 * theta + c2) * (c1 * theta + c2)$".

| | |
|---|---|
| **Question** | if f(theta) is {c1} times theta plus {c2} squared and theta is {theta} what is f(theta) ? |
| **Expression** | ({c1}*{theta}+{c2})*({c1}*{theta}+{c2}) |
| **Ex Question** | if f(theta) is 3 times theta plus 5 squared and theta is 0.5 what is f(theta) ? |
| **Ex Expression** | (3*0.5+5)*(3*0.5+5) |
| **Ex Answer** | 42.25 |

*Figure 3.* The first 6.036 gradient descent question template with a concrete example provided.

#### 4.1.3. Gradient Descent 2

The second gradient descent question also comes from week 4 exercises and is number 2.2. It is "f(theta) is (c1 times theta plus c2) squared and theta is theta and eta is eta what is theta after gd ? HINT use 2 times c1 times theta plus 2 times c2". For this question the values of theta, c1, c2, and eta all vary. The generated expression comes from "$theta - eta * 2 * c1 * (c1 * theta + c2)$".

#### 4.1.4. Filtering 1

This question comes from week 8 exercise 1.A.i, where one has to identify the length of an output vector produced by convolving an input vector $I$ by filter $F$, with stride equal to 1 and no padding. We used "I has length len_I and F has length len_F what is the length of the result of applying F to I" where the length of I and the length of F varied. The formula for the generated expression is "$len(I) - len(F) + 1$".

| | |
|---|---|
| **Question** | f(theta) is ({c1} times theta plus {c2}) squared and theta is {theta} and eta is {eta} what is theta after gd ? HINT use 2 times {c1} times theta plus 2 times {c2} . |
| **Expression** | {theta}-{eta}*2*{c1}*({c1}*{theta}+{c2}) |
| **Ex Question** | f(theta) is (3 times theta plus 7) squared and theta is 4 and eta is 0.1 what is theta after gd ? HINT use 2 times 3 times theta plus 2 times 7 . |
| **Ex Expression** | 4-0.1*2*3*(3*4+7) |
| **Ex Answer** | -7.4 |

*Figure 4.* The second 6.036 gradient descent question template with a concrete example provided.

| | |
|---|---|
| **Question** | I has length {len_I} and F has length {len_F} what is the length of the result of applying F to I ? |
| **Expression** | {len_I}-{len_F}+1 |
| **Ex Question** | I has length 5 and F has length 2 what is the length of the result of applying F to I ? |
| **Ex Expression** | 5-2+1 |
| **Ex Answer** | 4 |

*Figure 5.* The first 6.036 filtering question template with a concrete example provided.

#### 4.1.5. Filtering 2

The second filtering question comes from week 8 exercise 1.A.ii, where one now looks for actual values produced by convolving an input vector $I$ by filter $F$ with stride equal to 1 and no padding. In the actual exercises, the students are expected to return a python list of the full output list, but because we need a single numerical answer, we only as for the first value of that new list. The question format is "I is I and F is F what is the first value that results from applying F to I". For this the lists I and F vary. The result expression is generated by loop through the length of F and multiplying the elements at each index i from F and I. Each index is then summed.

| | |
|---|---|
| **Question** | I is {I} and F is {F} what is the first value that results from applying F to I ? |
| **Expression** | (I[0]*F[0])+(I[1]*F[1]) ... for length of F |
| **Ex Question** | I is [1, 2, 3, 4, 5] and F is [2, 0] what is the first value that results from applying F to I ? |
| **Ex Expression** | (1*2)+(2*0) |
| **Ex Answer** | 2 |

*Figure 6.* The second 6.036 filtering question template with a concrete example provided.

#### 4.1.6. RNN

The RNN based questions come from week 11's exercise's 2.A and 3.A/3.B, where one tries to identify the final output of an RNN at time $t$ where $s_t = w * s_{t-1} + x_t$. The prompt gives the equation, a value of $w$, an initial $s_0$ value and the various values of $x_t$ in a python list format like in the exercise questions. The somewhat tricky part of these questions comes from the fact that in order to solve them in a single expression, the model needs to learn how to nest. For example, if we ask for a value at $s_3$, the underlying expression will need to converted to $w * (w * (w * s_0 + x_1) + x_2) + x_3)$. We make sure to vary the time $t$ we look for so the model doesn't over train on just creating one set of expanded expression. The remaining RNN exercise questions were either text based question/answer pairs or those that required "pythonic" answers (not single numerical answer).

| | |
|---|---|
| **Question** | Consider a very simple RNN, defined by the following equation: s_t = w*s_t-1 + x_t. Given s_0 = {s_0} and w = {w} and x = {x} what is s_{i}? |
| **Expression** | w*s_t-1 + x_t recursively |
| **Ex Question** | Consider a very simple RNN, defined by the following equation: s_t = w*s_t-1 + x_t. Given s_0 = 3 and w = 1 and x = [1, 2] what is s_2? |
| **Ex Expression** | ((((3*1)+1)*1)+2) |
| **Ex Answer** | 6 |

*Figure 7.* The 6.036 RNN question template with a concrete example provided.

### 4.2. Benchmark with 57 subjects

The massive multitask benchmark from (Hendrycks et al., 2020) consists of multiple-choice questions from various branches of knowledge. The test spans subjects in the humanities, social sciences, hard sciences, and other areas that are important for some people to learn. There are 57 tasks (subjects) in total. The questions in the dataset were manually collected by graduate and undergraduate students from freely available sources online. These include practice questions for tests such as the Graduate Record Examination and the United States Medical Licensing Examination. It also includes questions designed for undergraduate courses and questions designed for readers of Oxford University Press books. Some tasks cover a subject, like psychology, but at a specific level of difficulty, such as "Elementary,""High School," "College," or "Professional." There are 15908 questions in total, split into a few-shot development set, a validation set, and a test set. The few-shot development set has 5 questions per subject, the validation set maybe used for selecting hyperparameters and is made of 1540 questions, and the test set has 14079 questions. Each subject contains 100 test examples at a minimum, and larger subjects contained over 1,000.

## 5. Results

### 5.1. 6.036 Questions

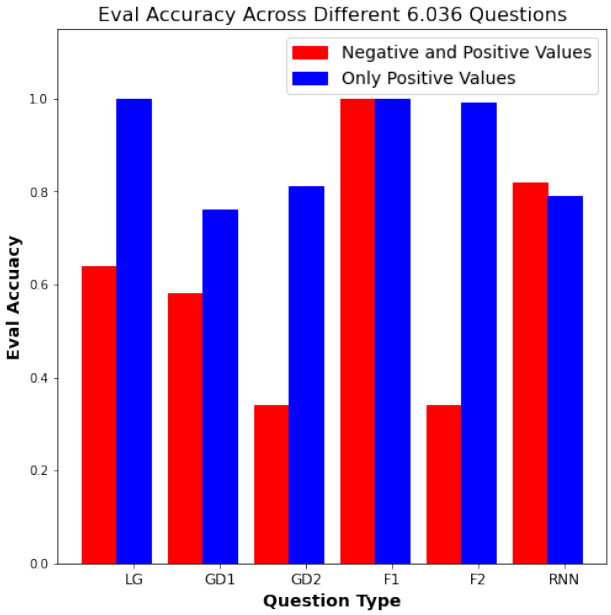


*Figure 8.* Performance on various types of 6.036 questions (such as Logistic Regression, Gradient Descent I II, CNN Filtering I & II, RNNs), using both datasets that include both positive and negative values, as well as a dataset only containing positive values in the question formulation.

| Neg | LG | GD1 | GD2 | F1 | F2 | RNN | Avg |
|---|---|---|---|---|---|---|---|
| +/- | 0.64 | 0.58 | 0.34 | 1.00 | 0.34 | 0.82 | 0.62 |
| - | 1.00 | 0.76 | 0.81 | 1.00 | 0.99 | 0.79 | 0.89 |

*Table 1.* Testing accuracies by question type for 6.036 questions, for both questions with positive and negative values, and questions with just postive values.

The overall accuracy increases from 0.62 to 0.89 when negative numbers were not considered as part of the body of the questions. The negative values affect two parts of the problem: both the expression that is used for the training data and the raw question that is parsed. We found that having negative numbers made it more likely to encounter a malformed expression (even when something such as (0-x) was placed in for a negative value) and that sometimes the negatives simply were not recognized. We believe these problems led to the decrease in accuracy when negative numbers are included.

#### 5.1.1. Logistic Regression

This logistic regression types of problem has an accuracy rate of 0.64 with negative values and 1.00 without negative values. Without the negatives, the formula is fairly straightforward as it involves multiplication and addition of values that are in the same quant cell position for every generated question. When the negatives are included the quant cell positions are shifted and the interpretation of the negatives becomes a problem as described for the overall score change, which is why we believe the two scores differ.

#### 5.1.2. Gradient Descent 1

Overall both gradient descent questions had accuracy rates lower than the average. The first gradient descent question had an accuracy rate of 0.58 with negatives and 0.76 without. This is probably on the lower side because of the repeated multiplication and the phrasing of squared and f(theta). However, refactoring the question could lead to a higher success rate. The question improves when negatives are removed for the same reasons stated in the above discussion on logistic regression performace.

#### 5.1.3. Gradient Descent 2

The second gradient descent question had an even lower accuracy rate. We believe this problem has a lower accuracy than the previous gradient descent problem since it has more variables (the addition of *eta*) and because it requires a derivative. Most likely, it also struggles with some of the problems mentioned for gradient descent 1 and could also be improved. The initial accuracy rate of 0.34 is greatly increased to 0.81 when removing negatives. Based on observations the more operations there are and the more negatives that there are the more likely a malformed expression error will occur resulting in an incorrect prediction. This occurs more often for this problem since there are more operations which explains the drastic improvement with the removal of negatives.

#### 5.1.4. Filtering 1

For the test set, all of the filtering questions produced a correct prediction (accuracy of 1.0). This is most likely because these questions used a simple formula that only required addition and subtraction with 2 variables and a constant. Additionally, since this question asks about lengths of lists there are no negative numbers which the model seemed to have a harder time with overall. These factors combined are why we believe the model had such a high success rate for this type of question. Since the initial accuracy rate was 1.0 and there were no negatives to remove(length is always non negative) the accuracy rate did not change with the removal of negatives.

#### 5.1.5. Filtering 2

This second filtering question did not perform as well as the first one. Its accuracy rate with negatives is 0.34. This is most likely because the problem contained multiple lists and being able to distinguish which numbers are needed for the calculation is a difficult task when the lists are not parsed as the same length. In order for the negatives to register they have to be inputed as ”negative 1” which then is counted as 2 different quant cells. However, when the negatives are removed the accuracy rate sky rockets to 0.99. This problem is strictly addition and multiplication with only 2 variables (the two lists) which is why we believe this accuracy rate is so high.

#### 5.1.6. RNN

The RNN questions did not contain negative numbers in the original version or in our generated questions, meaning there was not a dramatic increase in the accuracy rate when negative numbers are ”removed”. The accuracy rate went from 0.82 to 0.79 which is a small enough difference that can be attributed to the fact that network was trained with different data in each instance. We find that the accuracy is surprisingly impressive considering that the questions essentially created dynamic sets of nested expressions that vary based on the length of the provided input sequence, which is significantly more involved then learning a single question template and running new sets of variable values.

### 5.2. Heuristics: DistilBert QA

While we found this approach to give reasonable outputs in general, it does not prove very successful using our evaluation metric as the average result is worse than the random guessing benchmark of 0.25, as shown in Table 5.2. Our observation was that the model performs better when the multiple choice answers contain longer textual phrases, as opposed to numerical or boolean values. See Table 4 for examples of these questions and answers.

| Mean | Median | Min | Max |
|---|---|---|---|
| 0.2480 | 0.2487 | 0.1700 | 0.3714 |

*Table 2.* DistilBert QA with Heuristics accuracy results

### 5.3. Replacing GPT-3 with T5

We ran three variations of the T5 transformer model: the base, large and 3B, all of them in a zero-shot learning setting. We started by running the T5-base and then the T5-large model and lastly the T5-3B model and found that each of which was progressively better than the its predecessor. The T5 models were further trained on a collection of massive QA datasets like SQuAD and BoolQ

(Rajpurkar et al., 2018) (Clark et al., 2019). The results for this are shown below and an average success rate of 45% is achieved by T5-3B.

| Model | Mean | Median | Min | Max |
|---|---|---|---|---|
| T5-base | 0.3200 | 0.3179 | 0.1673 | 0.5148 |
| T5-large | 0.3844 | 0.3760 | 0.2185 | 0.7051 |
| T5-3B | 0.4542 | 0.4552 | 0.2018 | 0.7778 |

*Table 3.* Accuracy statistics on varying T5 Model sizes

# 6. Limitations

## 6.1. Limited Question Types

While we were answer six distinct question templates from 6.036, we were limited in the fact that all questions we could select from needed to be textual and have a single numerical answers. Unfortunately, that actually limits us from answering a majority of the questions from the 6.036 questions. There are really three other question templates that we could not solve with this approach.

1. First, we could not answer questions that relied on an input image as part of the questions. There are actually quite a few of these types of questions, whether is be for finding a classification based on nearest neighbors or determining the equation of a linear separator.

2. Another template of questions we could not answer were those whose answers were not single numerics. Particularly, a lot of 6.036 questions like the ask for solutions in the form of a python equation (maybe something like return correct backprop component).

3. Finally, there were also questions that were multiple choice that had text based options like "what would be the best activation function for this situation?".

## 6.2. Reinforcement Learning approach

One possibility to handle the issue with inputs of different modalities (visual, textual) is to have an outer Reinforcement Learning loop which selects which modality a question belongs to. The states would be the different setups for each modality (e.g. one state can be T5 with visual components, another state can be T5 with GNN etc). However, this wold require us to have implementations for different modalities and we currently have only the T5 + GNN setup. Future work can combine these setups as described.

# 7. Conclusion

In this work we looked at several approaches of Learning-to-Learn how to answer questions from several domains. We replaced GPT-3 with T5 in (Hendrycks et al., 2020) and reported results with different sizes of the T5 transformer. Furthermore, we curated a dataset which can be easily extended in size that contains 6 types of questions from 6.036 - Introduction to Machine Learning course offered on MITx. We have analyzed the performance for each question type of this dataset using the T5 + GNN approach and noted the non-trivial increase in performance when negative numbers are not used in the questions. Finally we report our results on some heuristic approaches and other less successful attempts, in the hopes that the deficiencies we have identified will be addressed in future work.

## 7.1. Project Links

The github repository with all of our project code can be found at https://github.com/sanjass/MultiTaskLearning

We also used the provided GCP credits for a Google Cloud Instance that can be found at https://console.cloud.google.com/home/dashboard?authuser=1&folder=&organizationId=&project=metalearning-293515

MIT 6.036 Exercise Questions can be found at https://lms.mitx.mit.edu/courses/course-v1:MITx+6.036+2020_Fall/course/

# References

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165*, 2020.

Chen, K., Xu, W., Cheng, X., Xiaochuan, Z., Zhang, Y., Song, L., Wang, T., Qi, Y., and Chu, W. Question directed graph attention network for numerical reasoning over text, 2020.

Clark, C., Lee, K., Chang, M.-W., Kwiatkowski, T., Collins, M., and Toutanova, K. Boolq: Exploring the surprising difficulty of natural yes/no questions, 2019.

Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

Kipf, T. N. and Welling, M. Semi-supervised classification with graph convolutional networks. *International Conference on Learning Representations*, 2017.

Koncel-Kedziorski, R., Roy, S., Amini, A., Kushman, N., and Hajishirzi, H. MAWPS: A math word problem repository. In *Proceedings of the 2016 Confer-*

| | |
|---|---|
| Question | Ani believes that her attitudes and behavior play a central role in what happens to her. Such a belief is likely to be associated with: |
| Context | a strong superego.low self-esteem.low self-efficacy.an internal locus of control. |
| Ground truth answer | an internal locus of control |
| Generated answer | internal locus of control. |
| Question | Which of the following contributes to vitamin B12 deficiency in older adults? |
| Context | Reduced secretion of intrinsic factor.Atrophic gastritis.Helicobacter pylori infection.All of the above |
| Ground truth answer | All of the above |
| Generated answer | Helicobacter pylori infection. |
| Question | Factor $36 - 9x^2$. |
| Context | $(6-3x)(6+3x).(-3x+6)(3x+6).9(2-x)(2+x).3(2+x^2)$ |
| Ground truth answer | $9(2-x)(2+x)$ |
| Generated answer | $9(2-x)(2+x)$. |
| Question | How many arithmetic sequences of consecutive odd integers sum to 240? |
| Context | 8.12.10.4. |
| Ground truth answer | 12 |
| Generated answer | 8. 12. 10. 4. |

*Table 4.* Examples of successful and unsuccessful predictions for the QA DistilBert with heuristics approach. The multiple choice answers are provided as context, separated by full-stop and a newline (not shown). While in most cases the generated answers are reasonable, the model fails to deal with multiple choice answers that are numerical only, predicting all numbers as the answer. Additionally, the model also seems to fail in understanding choices such as "All of the above", and instead seems to merely extract the probable sequence as the final answer.

*ence of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 1152–1157, San Diego, California, June 2016. Association for Computational Linguistics. doi: 10.18653/v1/N16-1136. URL https://www.aclweb.org/anthology/N16-1136.

Nichol, A., Achiam, J., and Schulman, J. On first-order meta-learning algorithms. *CoRR*, abs/1803.02999, 2018. URL http://arxiv.org/abs/1803.02999.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *arXiv preprint arXiv:1910.10683*, 2019.

Rajpurkar, P., Jia, R., and Liang, P. Know what you don't know: Unanswerable questions for squad, 2018.

Sanh, V., Debut, L., Chaumond, J., and Wolf, T. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter, 2020.

Wang, M., Zheng, D., Ye, Z., Gan, Q., Li, M., Song, X., Zhou, J., Ma, C., Yu, L., Gai, Y., et al. Deep Graph Library: A graph-centric, highly-performant package for graph neural networks. *arXiv preprint arXiv:1909.01315*, 2020.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., and Rush, A. ■ Transformers: State-of-the-art natural language processing. In *Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45, 2020a.

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Ma, C., Jernite, Y., Plu, J., Xu, C., Scao, T. L., Gugger, S., Drame, M., Lhoest, Q., and Rush, A. M. Huggingface's transformers: State-of-the-art natural language processing, 2020b.

Zhang, J., Wang, L., Lee, R. K.-W., Bin, Y., Wang, Y., Shao, J., and Lim, E.-P. Graph-to-tree learning for solving math word problems. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 3928–3937, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.362. URL https://www.aclweb.org/anthology/2020.acl-main.362.

Zheng, D., Wang, M., Gan, Q., Zhang, Z., and Karypis, G. Scalable graph neural networks with deep graph library. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 3521–3522, 2020.

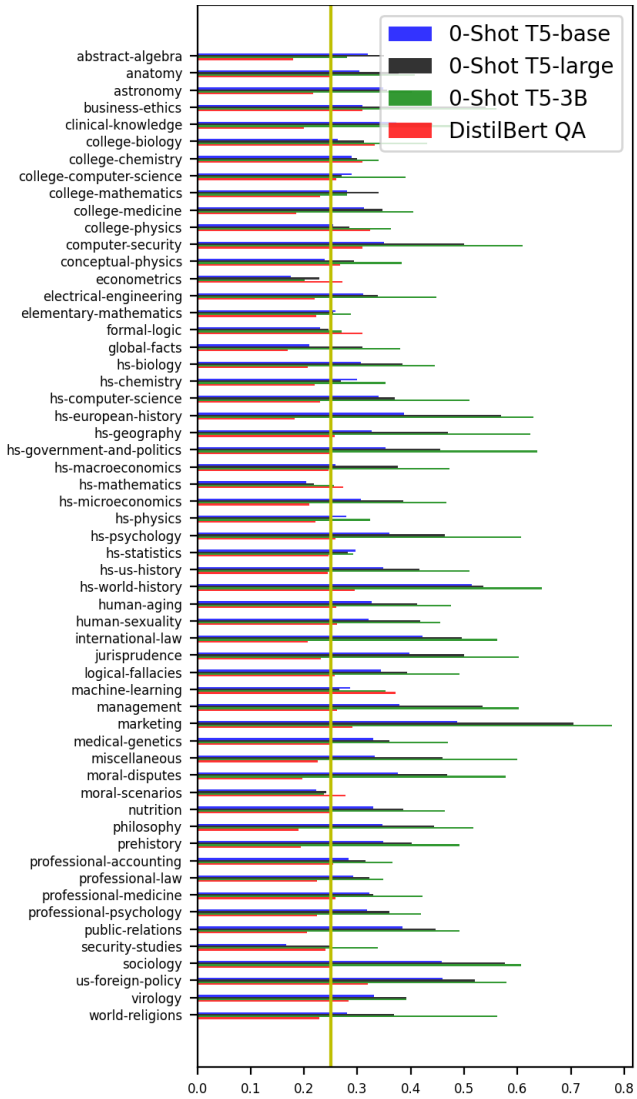


*Figure 9.* Performance of each model performance on each individual subject. T5-3B, the 3 billion parameter T5, is the best model by a nontrivial margin.